

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 <title>OperatorSense Live</title>
7 <style>
8 :root{
9   --bg:#0e1116; --panel:#161b22; --panel2:#1d2430; --border:#2a3340;
10   --text:#e6edf3; --muted:#9aa7b4; --accent:#3fb950; --accent2:#58a6ff;
11   --warn:#d29922; --err:#f85149;
12 }
13 *{box-sizing:border-box;}
14 body{
15   margin:0; font-family:"Segoe UI",Roboto,Helvetica,Arial,sans-serif;
16   background:var(--bg); color:var(--text); padding:16px;
17 }
18 h1{font-size:20px; margin:0 0 2px;}
19 .sub{color:var(--muted); font-size:13px; margin-bottom:16px;}
20 .bar{
21   display:flex; flex-wrap:wrap; gap:10px; align-items:center;
22   background:var(--panel); border:1px solid var(--border);
23   border-radius:10px; padding:12px; margin-bottom:16px;
24 }
25 button{
26   background:var(--accent2); color:#0b1622; border:none; border-radius:8px;
27   padding:9px 14px; font-size:14px; font-weight:600; cursor:pointer;
28 }
29 button:hover{filter:brightness(1.08);}
30 button.disabled{opacity:.45; cursor:not-allowed;}
31 button.ghost{background:transparent; color:var(--accent2); border:1px solid var(--border);}
32 button.danger{background:var(--err); color:#fff;}
33 button.go{background:var(--accent); color:#06210d;}
34 select{
35   background:var(--panel2); color:var(--text); border:1px solid var(--border);
36   border-radius:8px; padding:8px; font-size:14px;
37 }
38 .spacer{flex:1;}
39 .status{font-size:13px; color:var(--muted);}
40 .dot{display:inline-block; width:9px; height:9px; border-radius:50%; background:#555; margin-right:6px; vertical-align:middle;}
41 .dot.on{background:var(--accent); box-shadow:0 0 8px var(--accent);}
42 .dot.err{background:var(--err);}
43 .grid{display:grid; grid-template-columns:repeat(auto-fit,minmax(120px,1fr)); gap:12px; margin-bottom:16px;}
44 .card{background:var(--panel); border:1px solid var(--border); border-radius:10px; padding:14px;}
45 .card .label{color:var(--muted); font-size:12px; text-transform:uppercase; letter-spacing:.04em;}
46 .card .val{font-size:30px; font-weight:700; margin-top:4px; font-variant-numeric:tabular-nums;}
47 .card.big .val{font-size:52px; color:var(--accent);}
48 .card .unit{font-size:13px; color:var(--muted); font-weight:400;}
49 .panel{background:var(--panel); border:1px solid var(--border); border-radius:10px; padding:14px; margin-bottom:16px;}
50 .panel h2{font-size:14px; margin:0 0 10px; color:var(--muted); text-transform:uppercase; letter-spacing:.04em;}
51 canvas{width:100%; height:280px; display:block; background:var(--panel2); border-radius:8px;}
52 #rawlog{
53   font-family:Consolas,"Courier New",monospace; font-size:12px; line-height:1.5;
54   background:#0a0d12; border:1px solid var(--border); border-radius:8px;
55   height:180px; overflow:auto; padding:8px; color:#b9c4cf; white-space:pre;
56 }
57 .banner{
58   background:#3a2a08; border:1px solid var(--warn); color:#ffd98a;
59   border-radius:10px; padding:12px 14px; margin-bottom:16px; font-size:14px;
60 }
61 .banner.err{background:#3a0d0d; border-color:var(--err); color:#ffb3ad;}
62 .hidden{display:none;}
63 .hint{color:var(--muted); font-size:12px; margin-top:8px;}
64 a{color:var(--accent2);}
65 .decoded{font-family:Consolas,monospace; font-size:12px; color:var(--muted);}
66 .decoded b{color:var(--text);}
67 input{
68   background:var(--panel2); color:var(--text); border:1px solid var(--border);
69   border-radius:8px; padding:8px; font-size:13px; width:100%;
70 }
71 input.disabled{opacity:.45;}
72 .subpanel{margin-top:14px; border-top:1px solid var(--border); padding-top:12px;}
73 .row{display:flex; flex-wrap:wrap; gap:10px; align-items:center;}
74 .cfggrid{display:grid; grid-template-columns:repeat(auto-fit,minmax(230px,1fr)); gap:12px; margin:12px 0;}
75 .field{display:flex; flex-direction:column; gap:4px;}
76 .field > span{font-size:12px; color:var(--muted); text-transform:uppercase; letter-spacing:.04em;}
77 .field select{width:100%;}
78 .status.ok{color:var(--accent);}
79 .status.err{color:var(--err);}
80 .status.busy{color:var(--accent2);}
81 </style>
82 </head>

```

```

83 <body>
84   <h1>OperatorSense Live</h1>
85   <div class="sub">Reads the FTDI USB dongle directly in the browser (Web Serial). No PuTTY, no install.</div>
86
87   <div id="unsupported" class="banner err hidden">
88     ⚠ This browser doesn't support Web Serial. Please open this file in <b>Google Chrome</b> or
89     <b>Microsoft Edge</b> on a desktop PC.
90   </div>
91
92   <div class="bar">
93     <button id="connectBtn" class="go">⚡ Connect dongle</button>
94     <button id="disconnectBtn" disabled>Disconnect</button>
95     <label class="status">Baud
96       <select id="baud">
97         <option>9600</option>
98         <option>19200</option>
99         <option selected>38400</option>
100        <option>57600</option>
101        <option>115200</option>
102      </select>
103    </label>
104    <span class="status">8&nbsp;data&nbsp;&nbsp;&nbsp;No&nbsp;parity&nbsp;&nbsp;&nbsp;1&nbsp;stop</span>
105    <span class="spacer"></span>
106    <span class="status"><span id="dot" class="dot"></span><span id="statusText">Disconnected</span></span>
107  </div>
108
109  <div class="grid">
110    <div class="card big">
111      <div class="label">Current reading (raw ADC)</div>
112      <div class="val"><span id="cur"></span></div>
113      <div class="unit" id="ohms">Resistance: – &nbsp;<span style="opacity:.7">(vendor calibration, per ring address)</span></div>
114    </div>
115    <div class="card"><div class="label">Min</div><div class="val"><span id="min"></span></div></div>
116    <div class="card"><div class="label">Max</div><div class="val"><span id="max"></span></div></div>
117    <div class="card"><div class="label">Average</div><div class="val"><span id="avg"></span></div></div>
118    <div class="card"><div class="label">Sample rate</div><div class="val"><span id="interval"></span> <span class="unit">s/sam
119      ple</span></div><div class="unit"><span id="rate"></span> Hz</div></div>
120    <div class="card"><div class="label">Duration</div><div class="val"><span id="duration">0:00</span></div></div>
121    <div class="card"><div class="label">Recorded</div><div class="val"><span id="recCount">0</span> <span class="unit">samples<
122      /span></div></div>
123    <div class="card"><div class="label">Events</div><div class="val"><span id="evtCount">0</span></div></div>
124  </div>
125
126  <div class="panel">
127    <h2>Live signal</h2>
128    <canvas id="chart"></canvas>
129    <div class="decoded" id="decoded">Last packet: –</div>
130  </div>
131
132  <div class="panel">
133    <h2>Recording & export</h2>
134    <div style="display:flex; flex-wrap:wrap; gap:10px; align-items:center;">
135      <button id="recBtn" class="go" disabled>● Start recording</button>
136      <button id="markBtn" class="ghost" disabled>■ Mark event</button>
137      <button id="csvBtn" class="ghost" disabled>↓ Download CSV</button>
138      <button id="uploadBtn" class="ghost" disabled>↑ Send to server</button>
139      <button id="clearBtn" class="ghost" disabled>Clear recording</button>
140    </div>
141    <span class="hint">Recording captures every reading with a timestamp for CSV export. “Mark event” drops a labelled marker
142      (e.g. touch / release / stimulus) onto the graph and into the CSV. Watching the live graph does not require recording.
143    </span>
144  </div>
145
146  <div class="subpanel">
147    <div class="row">
148      <b style="font-size:13px;">Server upload</b>
149      <span id="uploadState" class="status">Not configured</span>
150      <span class="spacer"></span>
151      <button id="uploadCfgBtn" class="ghost">⚙ Server settings</button>
152    </div>
153
154    <div id="uploadCfg" class="hidden">
155      <div class="cfggrid">
156        <label class="field">
157          <span>Endpoint URL</span>
158          <input id="cfgEndpoint" type="url" placeholder="https://api.example.org/v1/eda/captures" autocomplete="off" />
159        </label>
160        <label class="field">
161          <span>Authentication</span>
162          <select id="cfgScheme">
163            <option value="bearer">Bearer token (Authorization)</option>
164            <option value="apikey">API key header</option>
165            <option value="none">None (open endpoint)</option>
166          </select>
167        </label>
168      </div>
169    </div>
170  </div>

```

```

161         </select>
162     </label>
163     <label class="field">
164         <span>API key / token</span>
165         <input id="cfgToken" type="password" placeholder="paste the key issued by your server" autocomplete="off" />
166     </label>
167     <label class="field">
168         <span>API key header name</span>
169         <input id="cfgHeader" type="text" placeholder="X-API-Key" autocomplete="off" />
170     </label>
171     <label class="field">
172         <span>Session label (optional)</span>
173         <input id="cfgLabel" type="text" placeholder="e.g. operator-07 / pilot-run-2" autocomplete="off" />
174     </label>
175 </div>
176 <div class="row">
177     <button id="cfgSaveBtn" class="go">Save settings</button>
178     <button id="cfgTestBtn" class="ghost">Test connection</button>
179     <button id="cfgForgetBtn" class="ghost">Forget credentials</button>
180 </div>
181 <div class="hint">
182     Nothing is sent anywhere until an endpoint is filled in here – the upload is wired up but
183     points at no server by default. Settings are stored in this browser only (localStorage);
184     <b>"Forget credentials"</b> wipes them. The payload is a single JSON <code>POST</code>
185     (schema documented at the top of the script). The server must return 2xx and allow CORS –
186     pages opened as <code>file://</code> send <code>Origin: null</code>, so serving this file over
187     <code>http://</code> is the easier path.
188 </div>
189 </div>
190 </div>
191 </div>
192
193 <div class="panel">
194     <h2>Raw serial log</h2>
195     <div id="rawlog"></div>
196     <div class="hint">Showing the last 200 lines as they arrive from the dongle.</div>
197 </div>
198
199 <script>
200 /* =====
201    ADC -> resistance conversion (vendor calibration).
202
203    Source: CALIBRATION.xlsx in this folder, measured by the device's developer.
204    Reference resistors were connected to each ring and the reported ADC code
205    noted, giving one table per ring:
206
207      R [kOhm]   CAFE (band)   CAD1 (no band)
208      0.55       --           --           <- below the detection floor
209      1           3           3
210      2.2         9           10
211      5.55        27          28
212      8.19        40          42
213      17.9        93          96
214      38.9        212         212
215      68.3        361         377
216      100.2       554         550
217
218    The ring address travels in every packet, so the matching table is picked
219    automatically; an unknown ring falls back to CALIBRATION_FALLBACK_RING.
220    Values are piecewise-linearly interpolated between table points and linearly
221    extrapolated outside them (the response is close to, but not exactly, linear).
222    Readings outside 1-100 kOhm are therefore estimates - the UI flags them as
223    "extrapolated" and every export carries an in-calibrated-range flag.
224
225    To add a further ring, append its address and its measured pairs below.
226    ===== */
227 let CONVERSION_ENABLED = true;
228
229 const CALIBRATION_TABLES = { // ring address -> [[adcCode, kOhm], ...] ascending
230     "CAFE": [[3,1],[9,2.2],[27,5.55],[40,8.19],[93,17.9],[212,38.9],[361,68.3],[554,100.2]],
231     "CAD1": [[3,1],[10,2.2],[28,5.55],[42,8.19],[96,17.9],[212,38.9],[377,68.3],[550,100.2]]
232 };
233 const CALIBRATION_FALLBACK_RING = "CAFE";
234
235 function calibrationFor(ringAddr){
236     const key = String(ringAddr || "").trim().toUpperCase();
237     return CALIBRATION_TABLES[key] || CALIBRATION_TABLES[CALIBRATION_FALLBACK_RING];
238 }
239
240 /* Raw ADC code -> resistance in ohms (null when it cannot be converted). */
241 function adcToOhms(adc, ringAddr){
242     if(adc === null || adc === undefined || Number.isNaN(adc)) return null;
243     const t = calibrationFor(ringAddr);

```

```

244   if(!t || t.length < 2) return null;
245   let i;
246   if(adc <= t[0][0]) i = 0; // extrapolate below the table
247   else if(adc >= t[t.length-1][0]) i = t.length - 2; // extrapolate above the table
248   else { i = 0; while(i < t.length - 2 && adc > t[i+1][0]) i++; }
249   const [a0, r0] = t[i], [a1, r1] = t[i+1];
250   const kOhm = r0 + (adc - a0) * (r1 - r0) / (a1 - a0);
251   return Math.max(0, kOhm * 1000);
252 }
253
254 /* True while the reading sits inside the measured calibration range. */
255 function adcInCalibratedRange(adc, ringAddr){
256   const t = calibrationFor(ringAddr);
257   return !!t && adc >= t[0][0] && adc <= t[t.length-1][0];
258 }
259
260 /* Skin conductance in microsiemens - the usual unit for EDA analysis. */
261 function ohmsToMicroSiemens(ohms){
262   return (ohms && ohms > 0) ? 1e6 / ohms : null;
263 }
264
265 /* =====
266   SERVER UPLOAD – fully wired, deliberately pointed at NO server.
267
268   Whoever takes this over only has to open "⚙ Server settings" in the
269   "Recording & export" panel and fill in the endpoint URL + API key; no code
270   change is needed. To ship a pre-filled build instead, put the values in
271   UPLOAD_DEFAULTS below (saved settings in the browser still win).
272
273   The server receives one JSON POST per capture:
274
275   {
276     "schemaVersion": "1.0",
277     "source": "OperatorSense Live (EDA ring, FTDI Web Serial)",
278     "sentAtIso": "2026-08-18T09:12:33.120Z",
279     "test": false, // true = "Test connection" ping, no samples
280     "capture": {
281       "captureId": "os-20260818T091120-4f3a", // stable per recording -> use it to de-duplicate
282       "label": "operator-07", // optional, from the settings form
283       "startedAtIso": "...", "endedAtIso": "...",
284       "durationMs": 61234, "sampleCount": 612, "eventCount": 3, "baudRate": 38400
285     },
286     "device": { "dongleAddr": "CEE3", "ringAddr": "CAFE", "usbVendorId": "0x0403" },
287     "signal": { "unit": "raw_adc", "ohmsAvailable": false },
288     "samples": [ { "epochMs": 1755506... , "elapsedMs": 0, "valueAdc": 1068, "ohms": null,
289                   "packetId": "D8730", "channel": "01", "dongleAddr": "CEE3",
290                   "ringAddr": "CAFE", "raw": "D8730:01[CEE3];CAFE;1068" }, ... ],
291     "events": [ { "epochMs": ..., "elapsedMs": ..., "label": "touch" }, ... ]
292   }
293
294   Any 2xx response counts as success; the recorded data is kept either way, so
295   a failed upload can be retried or exported to CSV instead.
296   ===== */
297 const UPLOAD_DEFAULTS = {
298   endpoint: "", // e.g. "https://api.example.org/v1/eda/captures"
299   scheme: "bearer", // "bearer" | "apikey" | "none"
300   token: "", // API key / bearer token
301   header: "X-API-Key", // header name used when scheme === "apikey"
302   label: "" // optional session label sent with every capture
303 };
304 const UPLOAD_STORAGE_KEY = "operatorsense.upload.v1";
305 const UPLOAD_TIMEOUT_MS = 30000;
306
307 /* ----- */
308 const FTDI_VENDOR_ID = 0x0403; // FT232R / FT232RQ
309
310 let port = null, reader = null, keepReading = false, textBuffer = "";
311 let chartData = []; // rolling {t, v} for the graph
312 const CHART_MAX = 600;
313 let recording = false;
314 let recorded = []; // {epoch, value, pktId, channel, dongle, ring, raw}
315 let events = []; // {epoch, label}
316 let rateStamps = []; // recent timestamps for rate calc
317 let lastValue = null;
318 let lastPacket = null; // last successfully parsed packet (device addresses)
319 let captureId = null; // identifies the current recording to the server
320 let firstSampleEpoch = null; // for duration + chart time axis
321 let lastSampleEpoch = null;
322 let dirty = false;
323
324 const $ = id => document.getElementById(id);
325
326 if(!("serial" in navigator)){

```

```

327     $(".unsupported").classList.remove("hidden");
328     $(".connectBtn").disabled = true;
329 }
330
331 /* ----- connection ----- */
332 $(".connectBtn").addEventListener("click", connect);
333 $(".disconnectBtn").addEventListener("click", disconnect);
334
335 async function connect(){
336     if(!("serial" in navigator)) return;
337     // Try with an FTDI filter first; if the user cancels, offer the unfiltered chooser.
338     try{
339         port = await navigator.serial.requestPort({ filters:[{ usbVendorId: FTDI_VENDOR_ID }] });
340     }catch(e){
341         try{
342             port = await navigator.serial.requestPort(); // no filter – show all serial ports
343         }catch(e2){
344             setStatus("No port selected", "muted");
345             return;
346         }
347     }
348     const baudRate = parseInt($(".baud").value, 10);
349     try{
350         await port.open({ baudRate, dataBits:8, parity:"none", stopBits:1, flowControl:"none" });
351     }catch(e){
352         setStatus("Open failed: " + e.message, "err");
353         port = null;
354         return;
355     }
356     // reset the live view for a fresh session (kept: recorded data + event marks)
357     chartData = []; rateStamps = []; lastValue = null;
358     firstSampleEpoch = null; lastSampleEpoch = null;
359     keepReading = true;
360     setConnectedUI(true);
361     setStatus("Connected – receiving", "on");
362     startRender();
363     readLoop();
364 }
365
366 async function readLoop(){
367     const decoder = new TextDecoder();
368     textBuffer = "";
369     while(port && port.readable && keepReading){
370         reader = port.readable.getReader();
371         try{
372             while(true){
373                 const { value, done } = await reader.read();
374                 if(done) break;
375                 if(value && value.length){
376                     textBuffer += decoder.decode(value, { stream:true });
377                     const lines = textBuffer.split(/\r\n|\r|\n/);
378                     textBuffer = lines.pop(); // keep the incomplete tail
379                     for(const ln of lines) handleLine(ln);
380                 }
381             }
382         }catch(e){
383             setStatus("Read error: " + e.message, "err");
384         }finally{
385             try{ reader.releaseLock(); }catch(_){}
386         }
387     }
388 }
389
390 async function disconnect(){
391     keepReading = false;
392     try{ if(reader) await reader.cancel(); }catch(_){}
393     try{ if(port) await port.close(); }catch(_){}
394     port = null; reader = null;
395     stopRender();
396     setConnectedUI(false);
397     setStatus("Disconnected", "muted");
398 }
399
400 /* ----- parsing ----- */
401 // Expected: D8730:01[CEE3];CAFE;1068 (value = integer after the last ';')
402 function parseLine(line){
403     line = line.trim();
404     if(!line) return null;
405     const parts = line.split(";");
406     if(parts.length < 2) return null;
407     const value = parseInt(parts[parts.length - 1].trim(), 10);
408     if(Number.isNaN(value)) return null;
409     let pktId = "", channel = "", dongle = "", ring = "";

```

```

410 const m = parts[0].match(/^(^:)+:(^[^:]+\|([^\|])+\|)$/);
411 if(m){ pktId = m[1]; channel = m[2]; dongle = m[3]; }
412 if(parts.length >= 3) ring = parts[parts.length - 2].trim();
413 return { value, pktId, channel, dongle, ring, raw: line };
414 }
415
416 function handleLine(line){
417     appendRaw(line);
418     const p = parseLine(line);
419     if(!p) return;
420     const now = Date.now();
421     if(firstSampleEpoch === null) firstSampleEpoch = now;
422     lastSampleEpoch = now;
423     lastValue = p.value;
424     chartData.push({ t: now, v: p.value });
425     if(chartData.length > CHART_MAX) chartData.shift();
426
427     rateStamps.push(now);
428     if(rateStamps.length > 50) rateStamps.shift();
429
430     if(recording){
431         recorded.push({ epoch: now, value: p.value, pktId:p.pktId, channel:p.channel,
432             dongle:p.dongle, ring:p.ring, raw:p.raw });
433     }
434     lastPacket = p;
435     $("decoded").innerHTML = "Last packet: " +
436         `pkt <b>${p.pktId}||"??"</b> · ch <b>${p.channel}||"??"</b> · ` +
437         `dongle <b>${p.dongle}||"??"</b> · ring <b>${p.ring}||"??"</b> · value <b>${p.value}</b>`;
438     dirty = true;
439 }
440
441 /* ----- raw log ----- */
442 let rawLines = [];
443 function appendRaw(line){
444     rawLines.push(line);
445     if(rawLines.length > 200) rawLines.shift();
446     const el = $("rawlog");
447     const atBottom = el.scrollTop + el.clientHeight >= el.scrollHeight - 4;
448     el.textContent = rawLines.join("\n");
449     if(atBottom) el.scrollTop = el.scrollHeight;
450 }
451
452 /* ----- stats + chart render loop ----- */
453 let renderTimer = null;
454 function startRender(){ if(!renderTimer) renderTimer = setInterval(render, 50); }
455 function stopRender(){ if(renderTimer){ clearInterval(renderTimer); renderTimer = null; } render(); }
456
457 function render(){
458     // big number always reflects latest
459     $("cur").textContent = lastValue === null ? "-" : lastValue;
460     if(CONVERSION_ENABLED && lastValue !== null){
461         const ring = lastPacket ? lastPacket.ring : null;
462         const ohms = adcToOhms(lastValue, ring);
463         if(ohms === null){
464             $("ohms").textContent = "Resistance: -";
465         }else{
466             const us = ohmsToMicroSiemens(ohms);
467             $("ohms").textContent = "Resistance: " + (ohms/1000).toFixed(2) + " kΩ · " +
468                 (us === null ? "-" : us.toFixed(2) + " μS") +
469                 (adcInCalibratedRange(lastValue, ring) ? "" : " (extrapolated)");
470         }
471     }
472     updateDuration();
473     if(!dirty){ updateRate(); return; }
474     dirty = false;
475
476     if(chartData.length){
477         const vals = chartData.map(d => d.v);
478         const mn = Math.min(...vals), mx = Math.max(...vals);
479         const sum = vals.reduce((a,b)=>a+b,0);
480         $("min").textContent = mn;
481         $("max").textContent = mx;
482         $("avg").textContent = Math.round(sum / vals.length);
483     }
484     $("recCount").textContent = recorded.length;
485     $("evtCount").textContent = events.length;
486     refreshUploadUi();
487     updateRate();
488     drawChart();
489 }
490
491 function updateDuration(){
492     if(firstSampleEpoch === null){ $("duration").textContent = "0:00"; return; }

```

```

493     const secs = Math.floor((lastSampleEpoch - firstSampleEpoch) / 1000);
494     const m = Math.floor(secs / 60), s = secs % 60;
495     $("duration").textContent = m + ":" + String(s).padStart(2, "0");
496 }
497
498 function updateRate(){
499     if(rateStamps.length >= 2){
500         const span = (rateStamps[rateStamps.length-1] - rateStamps[0]) / 1000;
501         if(span > 0){
502             const hz = (rateStamps.length - 1) / span;
503             $("rate").textContent = hz.toFixed(2);
504             $("interval").textContent = (1 / hz).toFixed(1);
505         }
506     }
507 }
508
509 /* ----- canvas chart ----- */
510 const canvas = $("chart");
511 const ctx = canvas.getContext("2d");
512 function sizeCanvas(){
513     const dpr = window.devicePixelRatio || 1;
514     const w = canvas.clientWidth, h = canvas.clientHeight;
515     canvas.width = w * dpr; canvas.height = h * dpr;
516     ctx.setTransform(dpr,0,0,dpr,0,0);
517     drawChart();
518 }
519 window.addEventListener("resize", sizeCanvas);
520
521 function drawChart(){
522     const w = canvas.clientWidth, h = canvas.clientHeight;
523     ctx.clearRect(0,0,w,h);
524     // grid
525     ctx.strokeStyle = "rgba(255,255,255,0.06)";
526     ctx.lineWidth = 1;
527     for(let i=0;i<=4;i++){ const y = (h/4)*i; ctx.beginPath(); ctx.moveTo(0,y); ctx.lineTo(w,y); ctx.stroke(); }
528     if(chartData.length < 2) return;
529
530     const vals = chartData.map(d => d.v);
531     let mn = Math.min(...vals), mx = Math.max(...vals);
532     if(mn === mx){ mn -= 1; mx += 1; }
533     const pad = (mx - mn) * 0.1; mn -= pad; mx += pad;
534
535     const tMin = chartData[0].t, tMax = chartData[chartData.length - 1].t;
536     const tSpan = (tMax - tMin) || 1;
537     const xForT = t => ((t - tMin) / tSpan) * w;
538     const yFor = v => h - ((v - mn) / (mx - mn)) * h;
539
540     // y labels
541     ctx.fillStyle = "rgba(255,255,255,0.45)";
542     ctx.font = "11px Consolas, monospace";
543     ctx.textAlign = "left";
544     ctx.fillText(Math.round(mx), 4, 14);
545     ctx.fillText(Math.round(mn), 4, h - 6);
546
547     // x time labels (elapsed seconds from the first sample of the session)
548     const t0 = firstSampleEpoch === null ? tMin : firstSampleEpoch;
549     ctx.fillText("t+" + ((tMin - t0) / 1000).toFixed(0) + "s", 30, h - 6);
550     ctx.textAlign = "right";
551     ctx.fillText("t+" + ((tMax - t0) / 1000).toFixed(0) + "s", w - 4, h - 6);
552     ctx.textAlign = "left";
553
554     // event markers (vertical dashed lines)
555     ctx.strokeStyle = "rgba(210,153,34,0.85)";
556     ctx.fillStyle = "rgba(210,153,34,0.95)";
557     ctx.setLineDash([4, 3]);
558     for(const ev of events){
559         if(ev.epoch < tMin || ev.epoch > tMax) continue;
560         const x = xForT(ev.epoch);
561         ctx.beginPath(); ctx.moveTo(x, 0); ctx.lineTo(x, h); ctx.stroke();
562         ctx.fillText(ev.label, x + 3, 24);
563     }
564     ctx.setLineDash([]);
565
566     // line
567     ctx.strokeStyle = "#3fb950";
568     ctx.lineWidth = 1.6;
569     ctx.beginPath();
570     for(let i=0;i<chartData.length;i++){
571         const x = xForT(chartData[i].t), y = yFor(chartData[i].v);
572         i===0 ? ctx.moveTo(x, y) : ctx.lineTo(x, y);
573     }
574     ctx.stroke();
575

```

```

576 // sample dots (only when sparse enough to stay readable)
577 if(chartData.length <= 200){
578   ctx.fillStyle = "#3fb950";
579   for(const d of chartData){
580     ctx.beginPath(); ctx.arc(xForT(d.t), yFor(d.v), 2, 0, Math.PI*2); ctx.fill();
581   }
582 }
583 // highlight latest point
584 const last = chartData[chartData.length - 1];
585 ctx.fillStyle = "#7ee787";
586 ctx.beginPath(); ctx.arc(xForT(last.t), yFor(last.v), 3.5, 0, Math.PI*2); ctx.fill();
587 }
588
589 /* ----- recording controls ----- */
590 $("recBtn").addEventListener("click", ()=>{
591   recording = !recording;
592   if(recording && !captureId) captureId = newCaptureId();
593   uploadNotice = null;
594   $("recBtn").textContent = recording ? "■ Stop recording" : "● Start recording";
595   $("recBtn").className = recording ? "danger" : "go";
596   refreshUploadUi();
597 });
598 $("markBtn").addEventListener("click", ()=>{
599   if(firstSampleEpoch === null) return;
600   const def = "Event " + (events.length + 1);
601   const input = prompt("Label for this event marker:", def);
602   if(input === null) return; // cancelled
603   events.push({ epoch: Date.now(), label: input.trim() || def });
604   $("evtCount").textContent = events.length;
605   dirty = true;
606 });
607 $("clearBtn").addEventListener("click", ()=>{
608   recorded = [];
609   events = [];
610   captureId = recording ? newCaptureId() : null;
611   uploadNotice = null;
612   $("recCount").textContent = "0";
613   $("evtCount").textContent = "0";
614   refreshUploadUi();
615 });
616 $("csvBtn").addEventListener("click", downloadCsv);
617
618 function csvNum(x, decimals){
619   return (x === null || x === undefined || Number.isNaN(x)) ? "" : x.toFixed(decimals);
620 }
621
622 function downloadCsv(){
623   if(!recorded.length){ alert("Nothing recorded yet. Press 'Start recording' first."); return; }
624   const t0 = recorded[0].epoch;
625   // attach each event marker to the nearest recorded sample (by time)
626   const evtByIndex = {};
627   for(const ev of events){
628     let best = 0, bestD = Infinity;
629     for(let i=0; i<recorded.length; i++){
630       const d = Math.abs(recorded[i].epoch - ev.epoch);
631       if(d < bestD){ bestD = d; best = i; }
632     }
633     evtByIndex[best] = evtByIndex[best] ? evtByIndex[best] + "|" + ev.label : ev.label;
634   }
635   const head = "timestamp_iso,epoch_ms,elapsed_ms,value_adc,resistance_ohm,conductance_us,in_calibrated_range,packet_id,channel,
636     dongle_addr,ring_addr,event,raw";
637   const rows = recorded.map((r, i) =>
638     [ new Date(r.epoch).toISOString(), r.epoch, r.epoch - t0, r.value,
639       csvNum(adcToOhms(r.value, r.ring), 1),
640       csvNum(ohmsToMicroSiemens(adcToOhms(r.value, r.ring)), 4),
641       adcInCalibratedRange(r.value, r.ring) ? 1 : 0,
642       r.pktId, r.channel, r.dongle, r.ring,
643       "'" + (evtByIndex[i] || "").replace(/"/g, "'") + "'",
644       "'" + r.raw.replace(/"/g, "'") + "'" ].join(",")
645   );
646   const blob = new Blob([head + "\n" + rows.join("\n")], { type:"text/csv" });
647   const url = URL.createObjectURL(blob);
648   const a = document.createElement("a");
649   const ts = new Date().toISOString().replace(/[:]/g, "-").slice(0,19);
650   a.href = url; a.download = `operatorsense_capture_${ts}.csv`;
651   a.click();
652   URL.revokeObjectURL(url);
653 }
654
655 /* ----- server upload -----
656 See the SERVER UPLOAD block at the top of this script for the payload schema
657 and for how to hand this over pre-configured. */

```

```

658 let uploadCfg      = Object.assign({}, UPLOAD_DEFAULTS);
659 let uploadBusy     = false;
660 let uploadNotice    = null;           // {text, kind} – result of the last send, until something changes
661 let lastPaintedState = null;
662
663 function newCaptureId(){
664     const ts = new Date().toISOString().replace(/[-:]/g, "").replace(/\./g, "");
665     return "os-" + ts + "-" + Math.random().toString(16).slice(2, 6);
666 }
667
668 /* --- settings persistence (browser-local; the key never leaves this machine) --- */
669 function loadUploadCfg(){
670     try{
671         const raw = localStorage.getItem(UPLOAD_STORAGE_KEY);
672         if(raw) uploadCfg = Object.assign({}, UPLOAD_DEFAULTS, JSON.parse(raw));
673     }catch(_){ /* private mode / file:// without storage – defaults stay in memory */ }
674     cfgToForm();
675 }
676
677 function persistUploadCfg(){
678     try{ localStorage.setItem(UPLOAD_STORAGE_KEY, JSON.stringify(uploadCfg)); return true; }
679     catch(_){ return false; } // in-memory only for this page load
680 }
681
682 function cfgToForm(){
683     $("cfgEndpoint").value = uploadCfg.endpoint;
684     $("cfgScheme").value   = uploadCfg.scheme;
685     $("cfgToken").value    = uploadCfg.token;
686     $("cfgHeader").value   = uploadCfg.header;
687     $("cfgLabel").value    = uploadCfg.label;
688     syncSchemeFields();
689 }
690
691 function formToCfg(){
692     uploadCfg = {
693         endpoint: $("cfgEndpoint").value.trim(),
694         scheme:   $("cfgScheme").value,
695         token:    $("cfgToken").value.trim(),
696         header:   ($("cfgHeader").value.trim() || "X-API-Key"),
697         label:    $("cfgLabel").value.trim()
698     };
699 }
700
701 function syncSchemeFields(){
702     const scheme = $("cfgScheme").value;
703     $("cfgToken").disabled = scheme === "none";
704     $("cfgHeader").disabled = scheme !== "apikey";
705 }
706
707 /* --- payload --- */
708 function knownRingTable(ringAddr){
709     const key = String(ringAddr || "").trim().toUpperCase();
710     return CALIBRATION_TABLES[key] ? key : CALIBRATION_FALLBACK_RING + " (fallback)";
711 }
712
713 function buildPayload(isTest){
714     const t0 = recorded.length ? recorded[0].epoch : null;
715     const tEnd = recorded.length ? recorded[recorded.length - 1].epoch : null;
716     const ohmsOf = (v, ring) => CONVERSION_ENABLED ? adcToOhms(v, ring) : null;
717     return {
718         schemaVersion: "1.0",
719         source: "OperatorSense Live (EDA ring, FTDI Web Serial)",
720         sentAtIso: new Date().toISOString(),
721         test: !!isTest,
722         capture: {
723             captureId: captureId || newCaptureId(),
724             label: uploadCfg.label || null,
725             startedAtIso: t0 === null ? null : new Date(t0).toISOString(),
726             endedAtIso:   tEnd === null ? null : new Date(tEnd).toISOString(),
727             durationMs:   t0 === null ? 0 : tEnd - t0,
728             sampleCount:  isTest ? 0 : recorded.length,
729             eventCount:   isTest ? 0 : events.length,
730             baudRate:     parseInt($("baud").value, 10)
731         },
732         device: {
733             dongleAddr: lastPacket ? lastPacket.dongle : null,
734             ringAddr:   lastPacket ? lastPacket.ring   : null,
735             usbVendorId: "0x" + FTDI_VENDOR_ID.toString(16).toUpperCase().padStart(4, "0")
736         },
737         signal: {
738             unit: "raw_adc",
739             ohmsAvailable: CONVERSION_ENABLED,
740             calibration: CONVERSION_ENABLED ? {
741                 source: "CALIBRATION.xlsx (device developer)",
742                 ringTable: knownRingTable(lastPacket && lastPacket.ring),
743                 rangeKOhm: [1, 100.2]
744             } : null
745         }
746     };

```

```

741     },
742     samples: isTest ? [] : recorded.map(r => ({
743         epochMs: r.epoch, elapsedMs: r.epoch - t0, valueAdc: r.value,
744         ohms: ohmsOf(r.value, r.ring),
745         microSiemens: ohmsToMicroSiemens(ohmsOf(r.value, r.ring)),
746         inCalibratedRange: adcInCalibratedRange(r.value, r.ring),
747         packetId: r.pktId, channel: r.channel, dongleAddr: r.dongle, ringAddr: r.ring, raw: r.raw
748     })),
749     events: isTest ? [] : events.map(ev => ({
750         epochMs: ev.epoch, elapsedMs: t0 === null ? null : ev.epoch - t0, label: ev.label
751     })))
752 };
753 }
754
755 function uploadHeaders(){
756     const h = { "Content-Type": "application/json" };
757     if(uploadCfg.scheme === "bearer" && uploadCfg.token) h["Authorization"] = "Bearer " + uploadCfg.token;
758     if(uploadCfg.scheme === "apikey" && uploadCfg.token) h[uploadCfg.header] = uploadCfg.token;
759     return h;
760 }
761
762 async function postPayload(payload){
763     const ctrl = new AbortController();
764     const timer = setTimeout(()=>ctrl.abort(), UPLOAD_TIMEOUT_MS);
765     try{
766         const res = await fetch(uploadCfg.endpoint, {
767             method: "POST", headers: uploadHeaders(),
768             body: JSON.stringify(payload), signal: ctrl.signal
769         });
770         if(!res.ok){
771             let detail = "";
772             try{ detail = (await res.text()).trim().slice(0, 160); }catch(_){}
773             throw new Error("HTTP " + res.status + " " + res.statusText + (detail ? " - " + detail : ""));
774         }
775         return res.status;
776     }catch(e){
777         if(e.name === "AbortError") throw new Error("Timed out after " + (UPLOAD_TIMEOUT_MS/1000) + " s - the server did not answer");
778         if(e.name === "TypeError") throw new Error("Network/CORS error - check the URL, that the server is reachable, and that it allows this page's origin (file:// pages send Origin: null)");
779         throw e;
780     }finally{ clearTimeout(timer); }
781 }
782
783 async function sendCapture(isTest){
784     if(uploadBusy) return;
785     if(!uploadCfg.endpoint){
786         openUploadCfg(true);
787         setUploadState("No endpoint configured - fill in the URL and press \u201cSave settings\u201d", "err");
788         return;
789     }
790     if(!isTest && !recorded.length){ alert("Nothing recorded yet. Press \u201cStart recording\u201d first."); return; }
791     if(uploadCfg.scheme !== "none" && !uploadCfg.token &&
792         !confirm("No API key is set. Send to the server without authentication?")) return;
793
794     uploadBusy = true; uploadNotice = null;
795     refreshUploadUi();
796     const n = isTest ? 0 : recorded.length;
797     setUploadState(isTest ? "Testing connection\u2026" : "Uploading " + n + " samples\u2026", "busy");
798     try{
799         const status = await postPayload(buildPayload(isTest));
800         uploadNotice = { kind:"ok",
801             text: (isTest ? "Test OK" : "Sent " + n + " samples") + " - HTTP " + status +
802                 " \u00b7 " + new Date().toLocaleTimeString() };
803     }catch(e){
804         uploadNotice = { kind:"err", text: (isTest ? "Test failed: " : "Upload failed: ") + e.message };
805     }finally{
806         uploadBusy = false;
807         refreshUploadUi();
808     }
809 }
810
811 /* --- ui state --- */
812 function setUploadState(text, kind){
813     const key = kind + "|" + text;
814     if(lastPaintedState === key) return;
815     lastPaintedState = key;
816     const el = $("uploadState");
817     el.textContent = text;
818     el.className = "status" + (kind && kind !== "muted" ? " " + kind : "");
819 }
820 function refreshUploadUi(){
821     const configured = !!uploadCfg.endpoint;

```

```

822     $("uploadBtn").disabled = uploadBusy || !configured || recorded.length === 0;
823     $("cfgTestBtn").disabled = uploadBusy;
824     if(uploadBusy) return; // the sender owns the text while it runs
825     if(uploadNotice){ setUploadState(uploadNotice.text, uploadNotice.kind); return; }
826     if(!configured){ setUploadState("Not configured – open Server settings", "muted"); return; }
827     setUploadState(recorded.length
828         ? "Ready – " + recorded.length + " samples queued"
829         : "Ready – nothing recorded yet", "muted");
830 }
831 function openUploadCfg(open){
832     $("uploadCfg").classList.toggle("hidden", !open);
833 }
834
835 /* --- wiring --- */
836 $("uploadBtn").addEventListener("click", ()=> sendCapture(false));
837 $("uploadCfgBtn").addEventListener("click", ()=> openUploadCfg($("uploadCfg").classList.contains("hidden")));
838 $("cfgScheme").addEventListener("change", syncSchemeFields);
839 $("cfgSaveBtn").addEventListener("click", ()=>{
840     formToCfg();
841     const stored = persistUploadCfg();
842     uploadNotice = { kind: uploadCfg.endpoint ? "ok" : "muted",
843         text: uploadCfg.endpoint
844             ? "Settings saved" + (stored ? "" : " (this page load only – browser storage unavailable)")
845             : "No endpoint set – uploads stay disabled" };
846     refreshUploadUi();
847 });
848 $("cfgTestBtn").addEventListener("click", ()=>{ formToCfg(); persistUploadCfg(); sendCapture(true); });
849 $("cfgForgetBtn").addEventListener("click", ()=>{
850     if(!confirm("Clear the stored endpoint and API key from this browser?")) return;
851     uploadCfg = Object.assign({}, UPLOAD_DEFAULTS);
852     try{ localStorage.removeItem(UPLOAD_STORAGE_KEY); }catch(_){}
853     cfgToForm();
854     uploadNotice = null;
855     refreshUploadUi();
856 });
857
858 loadUploadCfg();
859 refreshUploadUi();
860
861 /* ----- ui state ----- */
862 function setConnectedUI(on){
863     $("connectBtn").disabled = on;
864     $("disconnectBtn").disabled = !on;
865     $("baud").disabled = on;
866     $("recBtn").disabled = !on;
867     $("markBtn").disabled = !on;
868     $("csvBtn").disabled = !on;
869     $("clearBtn").disabled = !on;
870     if(!on){
871         recording = false;
872         $("recBtn").textContent = "● Start recording";
873         $("recBtn").className = "go";
874     }
875     refreshUploadUi();
876 }
877 function setStatus(text, kind){
878     $("statusText").textContent = text;
879     const dot = $("dot");
880     dot.className = "dot" + (kind === "on" ? " on" : kind === "err" ? " err" : "");
881 }
882
883 // auto-reconnect cleanup if the device is unplugged
884 if("serial" in navigator){
885     navigator.serial.addEventListener("disconnect", e => {
886         if(port && e.target === port){ disconnect(); setStatus("Dongle unplugged", "err"); }
887     });
888 }
889
890 sizeCanvas();
891 </script>
892 </body>
893 </html>

```